INTRODUCTION TO PROGRAMMING (2018-19)

Assignment 2

Write a C program (assume that its source file is named `calculate.c`) which reads from the input arithmetic expressions between integers, one on each line, until the end of the input, and prints, for each expression, its arithmetic result.

Expressions may contain "white space", that is space characters or tab characters (tab key), but not inside the numbers (their digits must be consecutive). There is no requirement to recognize and evaluate expressions that contain numbers, or whose evaluation requires intermediate results, that cannot be stored in the integer type `int`. If there is some syntax error in the expression, your program should detect it, print an error message, and continue with the next expression.

It is suggested that you develop your program gradually, with continuously increasing requirements, so that its debugging is easier. For this purpose, follow the steps presented below (the percentages in the parentheses are the weight of each step in the final evaluation of the program you will submit).

Step A (20%)

Expressions that contain additions and subtractions between numbers (without sign) should be recognized. The operations are performed from left to right. Of course, the result may also be a negative number. Even a simple integer is considered an expression, whose value is the number itself. A sample execution is shown below.

```
$ ./calculate
32+541
Result 1: 573
125 - 38 - 142 + 5
Result 2: -50
125 * 32
Result 3: Error!
  37-0
Result 4: 37
     125a + 3
Result 5: Error!
+ -
Result 6: Error!
678
Result 7:
678
^D
```

Step B (30%)

It should be possible for the terms recognized in the arithmetic expression with the additions and subtractions to be, apart from numbers, also products of integers. Recall that, in mathematics, in expressions that contain additions, subtractions and multiplications, the multiplications precede the additions and the subtractions, while the additions and subtractions are performed in order, from left to right. A sample execution is the following:

```
$ ./calculate
32 - 5*7
Result 1: -3
3*6*2*4 + 2*2*10  -7*8
Result 2: 128
^D
```

Step C (10%)

It should be possible for the expressions to also include numbers with a sign (`+` or `-`). Sample execution:

```
$ ./calculate
  32 +  -8
Result 1: 24
-25 + +7 - -9
Result 2: -9
-25 * -4 + 6 * -3 * 5
Result 3: 10
^D
```

Step D (30%)

It should be possible for the arithmetic expression to also include subexpressions within parentheses (at any depth). Sample execution:

```
$ ./calculate
5 * (3 + 7)
Result 1: 50
((-3 + 8) * ((7 + 2) * (5 - -6 - 8) + 3 * -5)) + (7 * (2 - 8))
Result 2: 18
^D
```

Step E (10%)

In case of an input error, a more precise diagnostic message about the kind of error should be printed. For the cases of errors that your program will recognize, you have relative freedom. Your error messages may be more (but not less) precise than those shown in the sample execution that follows.

```
$ ./calculate
35 27 - 4
Result 1: Unexpected character
32 * * 7
Result 2: Unexpected character
23 + 7 -
Result 3: Unexpected end of input
5 * (3 + 4
Result 4: Missing closing parenthesis
^D
```

Essentially, what is requested in this exercise is the implementation of a small part of the program `bc`, which is usually installed on Linux systems.

For details on the syntax of the allowed arithmetic expressions that are not clearly specified in the assignment description, you can either see how the program `bc` handles the specific case that troubles you and adopt its approach, or make some decision and follow it, documenting it clearly in the comments of your program.

For your convenience, the rules (in BNF - Backus-Naur Form) that determine when an arithmetic expression (`<expression>`) is syntactically acceptable are listed below. The symbol `|` stands for disjunction. Recall that white space is allowed within arithmetic expressions (something that is not described in the rules below), but not between the digits of a number.

```
<expression> ::= <term> | <expression> '+' <term> | <expression> '-' <term>
<term>       ::= <factor> | <factor> '*' <term>
<factor>     ::= <snumber> | '(' <expression> ')'
<snumber>    ::= <unumber> | '+' <unumber> | '-' <unumber>
<unumber>    ::= <digit> | <digit> <unumber>
<digit>      ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

In summary, the interpretation of the above rules is the following. An arithmetic expression is either a term, or a sequence of additions and subtractions of two or more terms. A term is either a factor or a product of two or more factors. A factor is either a number, or an arithmetic expression enclosed in parentheses. A number is either without a sign, or with a sign `+` or `-`. An unsigned number is a sequence of one or more digits (`0`, `1`, ..., `9`). Essentially, what the previous rules define is a *grammar* for the syntactically acceptable arithmetic expressions.

The submission of this exercise consists of uploading the source file `calculate.c` through eclass.

Note: In this exercise **the use of arrays (including strings) is strictly forbidden**. Also, the use of functions of the C input-output library that handle files is forbidden.